Deductive Verification of Choreographies

Sung-Shik Jongmans

Long-term research aim: (2010 onwards)

Develop new **methods** and **tools** to make **concurrent programming easier** (theory $\cap$ practice)

Long-term research aim: (2010 onwards)

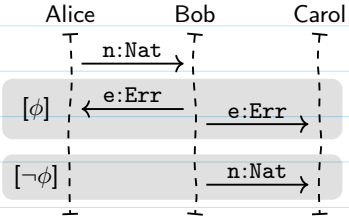
Develop new **methods** and **tools** to make **concurrent programming easier** (theory ∩ practice)

Keywords:

What:
Communication through **message passing**
(possibly shared memory)

Why:
Reasoning about **safety** and **liveness** of communication protocols is hard

How:
Static **analysis** and **synthesis** at compile-time

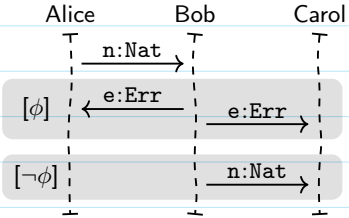


$\phi = \texttt{n<5} \wedge \texttt{n\%2=0}$

Idea: (sequence diagrams from a PL perspective)

1. Write a textual version of a sequence diagram
2. Do "something useful" with it (how much detail?)

```
A ⇀ B:Nat ;
∑{
  B ⇀ [A, C]:Err ,
  B ⇀ C:Nat
}
```



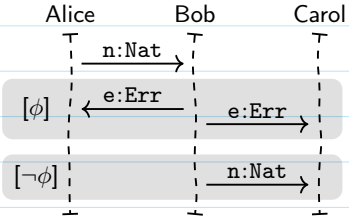
$\phi = \mathtt{n}<5 \wedge \mathtt{n}\%2=0$

Specification

Idea: (sequence diagrams from a PL perspective)

1. Write a textual version of a sequence diagram
2. Do "something useful" with it (how much detail?)

```
A →▷ B:Nat ;
∑{
  B →▷ [A, C]:Err ,
  B →▷ C:Nat
}
```



$\phi = \texttt{n<5} \wedge \texttt{n\%2=0}$

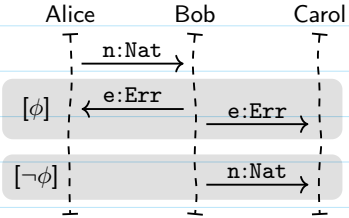
Analysis

```
B!n ;
if
  ϕ
then
  B?_:Err
```

```
A?n:Nat ;
if
  ϕ
then
  A!e ; C!e
else
  C!n
```

```
∑{
  B?_:Err ,
  B?m:Nat
}
```

```
A →▷ B:Nat ;
∑{
  B →▷ [A, C]:Err ,
  B →▷ C:Nat
}
```



$\phi = \mathtt{n<5} \wedge \mathtt{n\%2=0}$

```
A.n →▷ B.n:Nat ;
if
  A.ϕ ∧ B.ϕ
then
  B.e →▷ [A, C]._:Err
else
  B.n →▷ C.m:Nat
```

Analysis

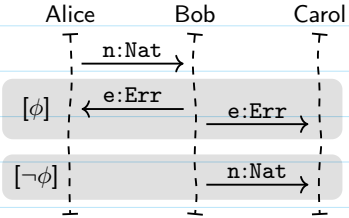
Implementation

```
B!n ;
if
  ϕ
then
  B?_:Err
```

```
A?n:Nat ;
if
  ϕ
then
  A!e ; C!e
else
  C!n
```

```
∑{
  B?_:Err ,
  B?m:Nat
}
```

```
A → B:Nat ;
∑{
  B → [A, C]:Err ,
  B → C:Nat
}
```



$\phi = \mathtt{n<5} \wedge \mathtt{n\%2=0}$

```
A.n → B.n:Nat ;
if
  A.φ ∧ B.φ
then
  B.e → [A, C]._:Err
else
  B.n → C.m:Nat
```

Analysis

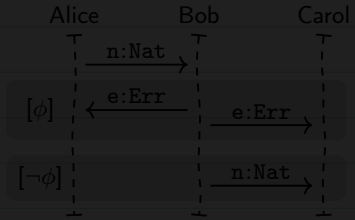
Synthesis

```
B!n ;
if
  φ
then
  B?_:Err
```

```
A?n:Nat ;
if
  φ
then
  A!e ; C!e
else
  C!n
```

```
∑{
  B?_:Err ,
  B?m:Nat
}
```

```
A ⇀ B:Nat ;
∑{
  B ⇀ [A, C]:Err ,
  B ⇀ C:Nat
}
```



$\phi = \mathtt{n}<5 \wedge \mathtt{n}\%2=0$

```
A.n ⇀ B.n:Nat ;
if
  A.ϕ ∧ B.ϕ
then
  B.e ⇀ [A, C]._:Err
else
  B.n ⇀ C.m:Nat
```

Analysis

Synthesis

```
B.n ;
ϕ
then
  B?_:Err
```

```
A?n:Nat ;
if
  ϕ
then
  A!e ; C!e
else
  C!n
```

```
∑{
  B?_:Err ,
  B?_:Nat
}
```

Multiparty session typing

[Honda et al., POPL'08]

Choreographic programming

[Carbone and Montesi, POPL'13]

```
A ⇀ B:Nat ;
∑{
  B ⇀ [A, C]:Err ,
  B ⇀ C:Nat
}
```

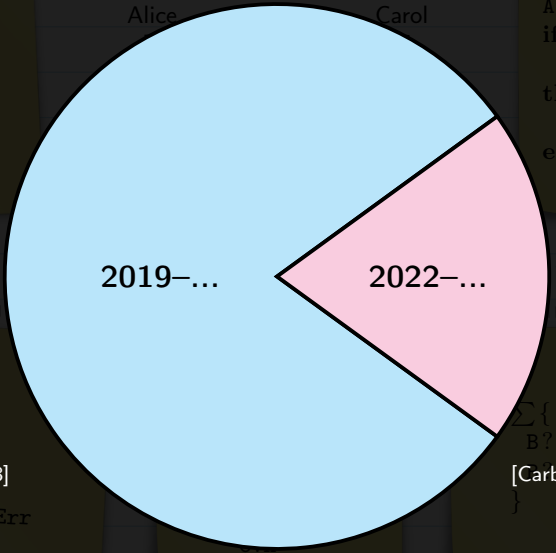
Alice

Carol

```
A.n ⇀ B.n:Nat ;
if
  A.φ ∧ B.φ
then
  B.e ⇀ [A, C]._:Err
else
  B.n ⇀ C.m:Nat
```

Analysis

Synthesis



Multiparty session typing

[Honda et al., POPL'08]

Choreographic programming

[Carbone and Montesi, POPL'13]

Rest of this talk

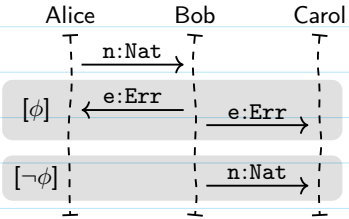
(joint work with Petra van den Bos, University of Twente)

ESOP'22
ESOP'25
FM'23
...

Synthesis

Choreographic programming

[Carbone and Montesi, POPL'13]



$\phi = \texttt{n<5} \wedge \texttt{n\%2=0}$

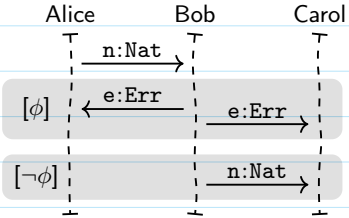
```
A.n ⇀ B.n:Nat ;
if
  A.ϕ ∧ B.ϕ
then
  B.e ⇀ [A, C]._:Err
else
  B.n ⇀ C.m:Nat
```

Synthesis

```
B!n ;
if
  ϕ
then
  B?_:Err
```

```
A?n:Nat ;
if
  ϕ
then
  A!e ; C!e
else
  C!n
```

```
∑{
  B?_:Err ,
  B?m:Nat
}
```



$\phi = \texttt{n<5} \wedge \texttt{n\%2=0}$

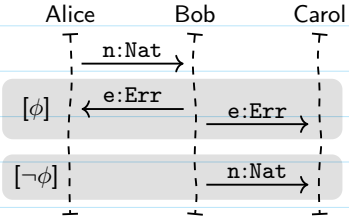
```
A.n ⇸ B.n:Nat ;
if
  A.ϕ ∧ B.ϕ
then
  B.e ⇸ [A, C]._:Err
else
  B.n ⇸ C.m:Nat
```

Synthesis (↑)

```
B!n ;
if
  ϕ
then
  B?_:Err
```

```
A?n:Nat ;
if
  ϕ
then
  A!e ; C!e
else
  C!n
```

```
∑{
  B?_:Err ,
  B?m:Nat
}
```



$\phi = \texttt{n<5} \wedge \texttt{n\%2=0}$

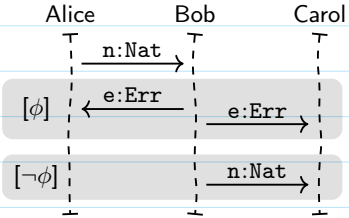
```
A.n ⇀ B.n:Nat ;
if
  A.ϕ ∧ B.ϕ
then
  B.e ⇀ [A, C]._:Err
else
  B.n ⇀ C.m:Nat
```

Synthesis (↑)

```
B!n ;
if
  ϕ
then
  B?_:Err
```

```
A?n:Nat ;
if
  ϕ
then
  A!e ; C!e
else
  C!n
```

```
∑{
  B?_:Err ,
  B?m:Nat
}
```



$\phi = \texttt{n<5} \wedge \texttt{n\%2=0}$

```
A.n → B.n:Nat ;
if
  A.ϕ ∧ B.ϕ
then
  B.e → [A, C]._:Err
else
  B.n → C.m:Nat
```

Synthesis (↑)

```
B!n ;
if
  ϕ
then
  B?_:Err
```

```
A?n:Nat ;
if
  ϕ
then
  A!e ; C!e
else
  C!n
```

```
∑{
  B?_:Err ,
  B?m:Nat
}
```

Theorem 1: ???

Theorem 2: ???

```
A.n ⇀ B.n:Nat ;
if
  A.φ ∧ B.φ
then
  B.e ⇀ [A, C]._:Err
else
  B.n ⇀ C.m:Nat
```

Synthesis (↑)

```
B!n ;
if
  φ
then
  B?_:Err
```

```
A?n:Nat ;
if
  φ
then
  A!e ; C!e
else
  C!n
```

```
∑{
  B?_:Err ,
  B?m:Nat
}
```

Theorem 1: $\mathrm{df}(G)$

Theorem 2: ???

```
A.n ⇀ B.n:Nat ;
if
  A.φ ∧ B.φ
then
  B.e ⇀ [A, C]._:Err
else
  B.n ⇀ C.m:Nat
```

Synthesis (↑)

```
B!n ;
if
  φ
then
  B?_:Err
```

```
A?n:Nat ;
if
  φ
then
  A!e ; C!e
else
  C!n
```

```
∑{
  B?_:Err ,
  B?m:Nat
}
```

Theorem 1: $\mathrm{df}(G)$

Theorem 2: $G \sim \prod\{G \upharpoonright r_1, \ldots, G \upharpoonright r_n\}$

(crux: $\upharpoonright$ is a partial function)

```
A.n →▷ B.n:Nat ;
if
  A.φ ∧ B.φ
then
  B.e →▷ [A, C]._:Err
else
  B.n →▷ C.m:Nat
```

Synthesis ($\upharpoonright$)

```
B!n ;
if
  φ
then
  B?_:Err
```

```
A?n:Nat ;
if
  φ
then
  A!e ; C!e
else
  C!n
```

```
∑{
  B?_:Err ,
  B?m:Nat
}
```

Theorem 1: $\mathrm{df}(G)$

Theorem 2: $G \sim \prod\{G \upharpoonright r_1, \ldots, G \upharpoonright r_n\}$

(crux: $\upharpoonright$ is a partial function)

```
A.n ↣ B.n:Nat ;
if
  A.φ ∧ B.φ
then
  B.e ↣ [A, ~~e~~]._:Err
else
  B.n ↣ C.m:Nat
```

Synthesis ($\upharpoonright$)

```
B!n ;
if
  φ
then
  B?_:Err
```

```
A?n:Nat ;
if
  φ
then
  A!e ; ~~e~~
else
  C!n
```

(syntactic check)

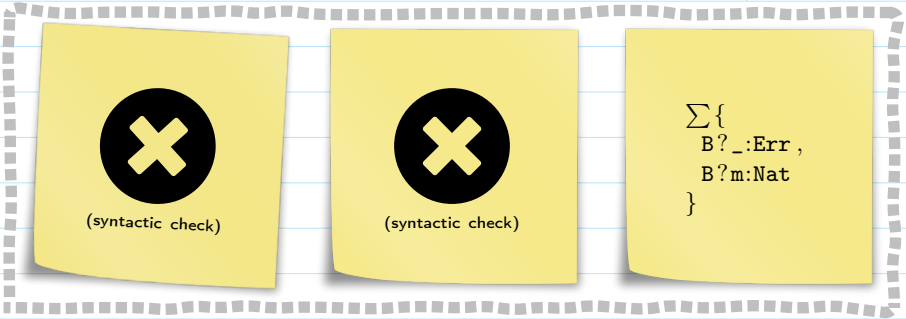
Theorem 1: $\mathrm{df}(G)$

Theorem 2: $G \sim \prod\{G \upharpoonright r_1, \ldots, G \upharpoonright r_n\}$

(crux: $\upharpoonright$ is a partial function)

```
A.n ⇀ B.n:Nat ;
if
  A.n<5 ∧ B.n%2=0
then
  B.e ⇀ [A, C]._:Err
else
  B.n ⇀ C.m:Nat
```

Synthesis ($\upharpoonright$)



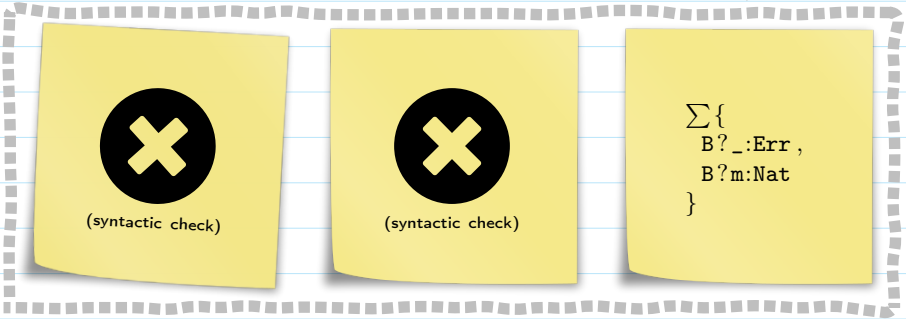
Theorem 1: $\mathrm{df}(G)$

Theorem 2: $G \sim \prod\{G \upharpoonright r_1, \ldots, G \upharpoonright r_n\}$

(crux: $\upharpoonright$ is a partial function)

```
A.n ⇸ B.n:Nat ;
if
  A.n<5 /\ B.n<8-3
then
  B.e ⇸ [A, C]._:Err
else
  B.n ⇸ C.m:Nat
```

Synthesis ($\hat{\upharpoonright}$)



Approach:

1. Widen definition of $\upharpoonright$
2. Strengthen premise of theorems

[ESOP'22, FM'23]

Theorem 1: $\mathrm{df}(G)$

Theorem 2: $G \sim \prod\{G \upharpoonright r_1, \ldots, G \upharpoonright r_n\}$

(crux: $\upharpoonright$ is a partial function)

Synthesis ($\hat{\upharpoonright}$)

(semantic check)

(semantic check)

```
∑{
  B?_:Err,
  B?m:Nat
}
```

Approach:

1. Widen $\boxed{\text{definition of } \upharpoonright}$
2. Strengthen premise of theorems

[ESOP'22, FM'23]

Theorem 1: $\mathrm{df}(G)$

Theorem 2: $G \sim \prod\{G \hat{\upharpoonright} r_1, \ldots, G \hat{\upharpoonright} r_n\}$

(crux: $\hat{\upharpoonright}$ is a partial function)

Synthesis ($\hat{\upharpoonright}$)

```
B!n;
if
  n<5
then
  B?_:Err
```

```
A?n:Nat;
if
  n<8-3
then
  A!e;C!e
else
  C!n
```

```
∑{
  B?_:Err,
  B?m:Nat
}
```

Premise: $\top \equiv \mathrm{wp}(G, \top)$

Theorem 1: $\mathrm{df}(G)$

Theorem 2: $G \sim \prod\{G \hat{\restriction} r_1, \ldots, G \hat{\restriction} r_n\}$

(crux: $\hat{\restriction}$ is a partial function)

Approach:

1. Widen definition of $\restriction$
2. Strengthen $\boxed{\text{premise of theorems}}$

[ESOP'22, FM'23]

Synthesis ($\hat{\restriction}$)

```
B!n;
if
  n<5
then
  B?_:Err
```

```
A?n:Nat;
if
  n<8-3
then
  A!e;C!e
else
  C!n
```

```
∑{
  B?_:Err,
  B?m:Nat
}
```

Premise: $\phi_{\text{pre}} = \text{wp}(G, \phi_{\text{post}}) \wedge \ldots$

Theorem 1: $\text{df}(G)$

Theorem 2: $G \sim \prod\{G \hat{\restriction} r_1, \ldots, G \hat{\restriction} r_n\}$

(crux: $\hat{\restriction}$ is a partial function)

Approach:

1. Widen definition of $\restriction$
2. Strengthen premise of theorems

[ESOP'22, FM'23]

Synthesis ($\hat{\restriction}$)

```
B!n;
if
  n<5
then
  B?_:Err
```

```
A?n:Nat;
if
  n%2=0
then
  A!e;C!e
else
  C!n
```

```
∑{
  B?_:Err,
  B?m:Nat
}
```

Weakest preconditions:

$$\mathrm{wp}(q.y := e, \phi) = \phi[e/q.y]$$

$$\mathrm{wp}(p.e \rightarrowtriangle q.y, \phi) = \phi[e/q.y]$$

$$\mathrm{wp}(G_1 \,;\, G_2, \phi) = \mathrm{wp}(G_1, \mathrm{wp}(G_2, \phi))$$ (surprisingly complicated)

Easy:

$$\frac{G_1 \xrightarrow{\alpha} G_1'}{G_1 \,;\, G_2 \xrightarrow{\alpha} G_1' \,;\, G_2}$$

Weakest preconditions:

$$\begin{aligned}
\mathrm{wp}(q.y := e, \phi) &= \phi[e/q.y] \\
\mathrm{wp}(p.e \rightarrowtriangle q.y, \phi) &= \phi[e/q.y] \\
\mathrm{wp}(G_1 \,;\, G_2, \phi) &= \mathrm{wp}(G_1, \mathrm{wp}(G_2, \phi)) \quad \text{(surprisingly complicated)}
\end{aligned}$$

Also easy:

$$\frac{G_1 \downarrow \quad G_2 \xrightarrow{\alpha} G_2'}{G_1 \,;\, G_2 \xrightarrow{\alpha} G_2'}$$

Weakest preconditions:

$$\begin{aligned}
\mathrm{wp}(q.y := e, \phi) &= \phi[e/q.y] \\
\mathrm{wp}(p.e \rightarrowtriangle q.y, \phi) &= \phi[e/q.y] \\
\mathrm{wp}(G_1 \,;\, G_2, \phi) &= \mathrm{wp}(G_1, \mathrm{wp}(G_2, \phi)) \quad \text{(surprisingly complicated)}
\end{aligned}$$

Hard:

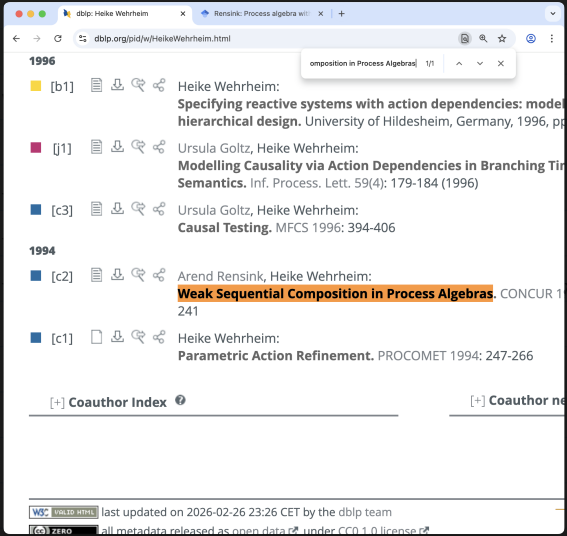
$$\frac{G_1 \cap \alpha = \emptyset \qquad G_2 \xrightarrow{\alpha} G_2'}{G_1 \,;\, G_2 \xrightarrow{\alpha} G_1 \,;\, G_2'}$$

(weak sequencing)

Weakest preconditions:

$$\begin{aligned}
\mathrm{wp}(q.y := e, \phi) &= \phi[e/q.y] \\
\mathrm{wp}(p.e \rightarrowtriangle q.y, \phi) &= \phi[e/q.y] \\
\mathrm{wp}(G_1 \,;\, G_2, \phi) &= \mathrm{wp}(G_1, \mathrm{wp}(G_2, \phi))
\end{aligned}$$

(surprisingly complicated)

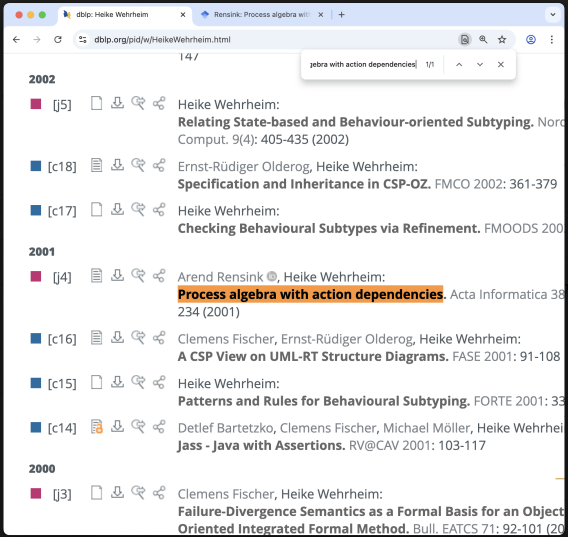


Hard:

$$\frac{G_1 \cap \alpha = \emptyset \qquad G_2 \xrightarrow{\alpha} G_2'}{G_1 \,;\, G_2 \xrightarrow{\alpha} G_1 \,;\, G_2'}$$

(weak sequencing)

$\mathrm{wp}(G_2, \phi))$ (surprisingly complicated)

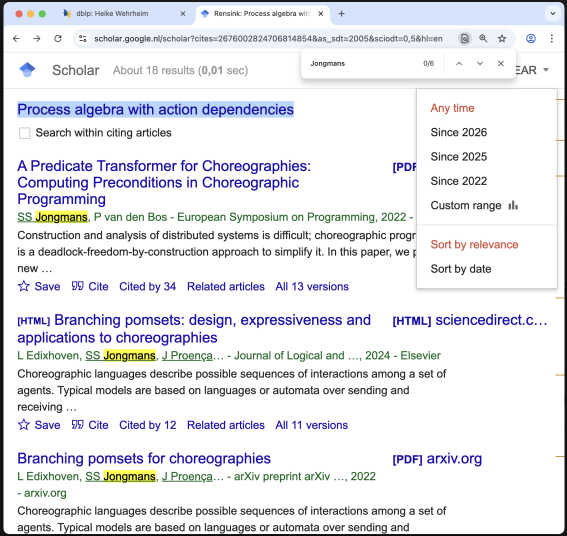


Hard:

$$\frac{G_1 \cap \alpha = \emptyset \qquad G_2 \xrightarrow{\alpha} G_2'}{G_1 \,;\, G_2 \xrightarrow{\alpha} G_1 \,;\, G_2'}$$

(weak sequencing)

$\mathrm{vp}(G_2, \phi))$ (surprisingly complicated)



Hard:

$$\frac{G_1 \cap \alpha = \emptyset \qquad G_2 \xrightarrow{\alpha} G_2'}{G_1 \,;\, G_2 \xrightarrow{\alpha} G_1 \,;\, G_2'}$$

(weak sequencing)

$\mathrm{vp}(G_2, \phi))$ (surprisingly complicated)

Hard:

$$\frac{G_1 \cap \alpha = \emptyset \qquad G_2 \xrightarrow{\alpha} G_2'}{G_1 \,;\, G_2 \xrightarrow{\alpha} G_1 \,;\, G_2'}$$

(weak sequencing)

Weakest preconditions:

$$\begin{aligned}
\mathrm{wp}(q.y := e, \phi) &= \phi[e/q.y] \\
\mathrm{wp}(p.e \rightarrowtriangle q.y, \phi) &= \phi[e/q.y] \\
\mathrm{wp}(G_1 \,;\, G_2, \phi) &= \mathrm{wp}(G_1, \mathrm{wp}(G_2, \phi))
\end{aligned}$$

(surprisingly complicated)

Hard:

$$\frac{G_1 \cap \alpha = \emptyset \qquad G_2 \xrightarrow{\alpha} G_2'}{G_1 \,;\, G_2 \xrightarrow{\alpha} G_1 \,;\, G_2'}$$

(weak sequencing)

Weakest preconditions:

$$\mathrm{wp}(q.y := e, \phi) = \phi[e/q.y]$$

$$\mathrm{wp}(p.e \rightarrowtriangle q.y, \phi) = \phi[e/q.y]$$

$$\mathrm{wp}(G_1 \,;\, G_2, \phi) = \mathrm{wp}(G_1, \mathrm{wp}(G_2, \phi))$$ (surprisingly complicated)

$$\mathrm{wp}(\textbf{if } \bigwedge\{e_r\}_{r \in R}\, G_1\, G_2) = \bigwedge\{\ \ e_r\}_{r \in R} \Rightarrow \mathrm{wp}(G_1, \phi) \wedge$$

$$\bigwedge\{\neg e_r\}_{r \in R} \Rightarrow \mathrm{wp}(G_2, \phi) \wedge$$

$$\bigwedge\{e_{r_1} \Rightarrow e_{r_2}\}_{r_1, r_2 \in R}$$ ("unanimity")

Hard:

$$\frac{G_1 \cap \alpha = \emptyset \qquad G_2 \xrightarrow{\alpha} G_2'}{G_1 \,;\, G_2 \xrightarrow{\alpha} G_1 \,;\, G_2'}$$

(weak sequencing)

Weakest preconditions:

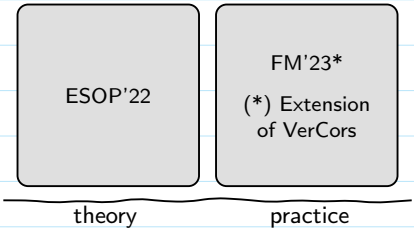
$$\mathrm{wp}(q.y := e, \phi) = \phi[e/q.y]$$

$$\mathrm{wp}(p.e \rightarrowtriangle q.y, \phi) = \phi[e/q.y]$$

$$\mathrm{wp}(G_1 \,;\, G_2, \phi) = \mathrm{wp}(G_1, \mathrm{wp}(G_2, \phi))$$ (surprisingly complicated)

$$\mathrm{wp}(\mathbf{if}\ \bigwedge\{e_r\}_{r \in R}\ G_1\ G_2) = \bigwedge\{\ \ e_r\}_{r \in R} \Rightarrow \mathrm{wp}(G_1, \phi) \wedge$$
$$\bigwedge\{\neg e_r\}_{r \in R} \Rightarrow \mathrm{wp}(G_2, \phi) \wedge$$
$$\bigwedge\{e_{r_1} \Rightarrow e_{r_2}\}_{r_1, r_2 \in R}$$ ("unanimity")

$$\mathrm{wp}(\mathbf{while}\ \bigwedge\{e_r\}_{r \in R}\ \{\phi_{\mathrm{inv}}\}\ G) = \ldots$$ (similar to **if**)



Static synthesis [Carbone and Montesi, POPL'13]

Problem space (deductive verification of choreographies)

ESOP'22

Graversen et al., COORDINATION'23

FM'23*

(*) Extension of VerCors

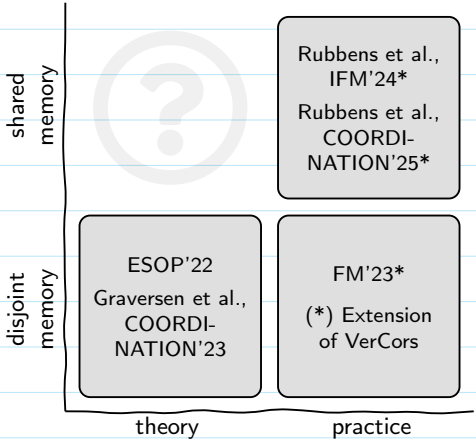
theory

practice

Static synthesis

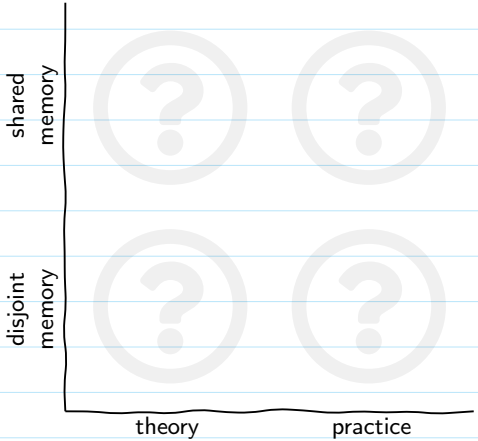
[Carbone and Montesi, POPL'13]

Problem space (deductive verification of choreographies)



Static synthesis
[Carbone and Montesi, POPL'13]

Problem space (deductive verification of choreographies)



Dynamic synthesis [ESOP'25] (event-driven)

Problem space (deductive verification of choreographies)